ch6 - Pointers: Notes

Exercise 6.1-6.3: ADD-friendly study notes

About These Notes

Short, scannable notes for Go Chapter 6 pointer exercises. Designed to be read in small chunks - one section at a time. No walls of text.

Covers: Exercise 6.1 (Person struct: value vs pointer), Exercise 6.2 (slice pointers: UpdateSlice vs GrowSlice), Exercise 6.3 (GC tuning with 10 million entries).

Exercise 6.1 - Person Struct: Value vs Pointer

What it does: Creates a Person struct (FirstName, LastName, Age) with two constructor functions:

- MakePerson -> returns a Person value (a copy)
- MakePersonPointer -> returns a *Person pointer (an address)

Key Difference

	Value Person	Pointer *Person
What's copied	The whole struct	Just the address (8 bytes)
Memory	New copy each time	One struct, many pointers
Changes to copy	Don't affect original	Affect the original

How to Call

```
p1 := MakePerson("Bertin", "Dreyer", 28)      // value
p2 := MakePersonPointer("Bertin", "Dreyer", 28) // pointer

println(p1.FirstName, p1.LastName, p1.Age)
println(p2.FirstName,    p2.LastName,    p2.Age)           //    Go
auto-dereferences p2
```

When to Use Which

- Use a value when you want an independent copy.

- Use a pointer when you want to share one struct or avoid copying large data.

Exercise 6.2 - Slice Pointers: UpdateSlice vs GrowSlice

What it does: Two functions that take a []string:

- UpdateSlice(slice, s) - changes the last element in place.
- GrowSlice(slice, s) - appends a new element.

The Gotcha: Slices Are Headers, Not Data

A slice header has 3 parts:

1. Pointer to the underlying array
2. Length
3. Capacity

When you pass a slice to a function, the header is copied, but the pointer still points to the same underlying array.

UpdateSlice (Works on Caller's Data)

Because the copied header still points to the same array, the caller sees the change.

```
func UpdateSlice(slice []string, s string) {
    slice[len(slice)-1] = s   // modifies the shared array
}
```

GrowSlice (Does NOT Persist to Caller)

If append needs more room, it allocates a new array and updates the local header. The caller's header still points to the old array, so the caller doesn't see the new element.

```
func GrowSlice(slice []string, s string) {
    slice = append(slice, s)   // may create a NEW array
}
```

Fix: Reassign in main

```
words = GrowSlice(words, "date")
```

Output You'll See

```
Before UpdateSlice: [apple banana carrot]
After UpdateSlice:  [apple banana cherry]   <- changed!

Before GrowSlice:   [apple banana cherry]
After GrowSlice:    [apple banana cherry]   <- NOT changed in main!
```

Exercise 6.3 - GC Tuning with 10 Million Entries

What it does: Builds a []Person with 10,000,000 entries, times it, and prints how many GC runs happened.

Why It Matters

Every time append reallocates the slice, it copies old data to a new array. The old array becomes garbage. More garbage = more GC runs = slower.

GOGC

Controls how often GC runs. GOGC = percent of heap growth allowed before triggering GC. Default = 100.

- Higher GOGC -> fewer GC runs -> usually faster, but uses more memory.
- Lower GOGC -> more GC runs -> usually slower, but keeps memory lower.

How to Test

```
# Default (GOGC=100)
go run .

# Less frequent GC
$env:GOGC="500"; go run .

# More frequent GC
$env:GOGC="10"; go run .
```

Results (This Machine, No Pre-allocation)

GOGC	Time	GC Runs
100 (default)	1.63s	15
500	1.20s	4 <- fastest
10	1.81s	30 <- slowest

> Takeaway: Higher GOGC = faster here because fewer GC pauses. But it uses more

> memory between collections.

> Pro tip: Pre-allocate capacity to avoid reallocations entirely:
>
> ```
> people := make([]Person, 0, 10_000_000)
> ```
>
> This usually makes it faster than any GOGC setting because there's no copying at all.

Quick Reference

- Struct value = full copy. Struct pointer = just an address.
- Slice header is copied by value, but points to shared data.
- Append may reallocate - caller won't see new elements unless you reassign.
- GOGC trades memory for GC frequency. Higher = fewer GCs, usually faster.